

Mostafa Mahmoud

Software Engineer

+201014414536 mostafasensei106@gmail.com Mostafa Mahmoud Personal Website

MostafaSensei106

WORK EXPERIENCE

Flutter Developer [Volunteer]

Aug 2024 – Aug 2024

Lothga - Task Based

Jeddah, Saudi Arabia, SA

During the development of the "LothgaTablet" project:

- Developed the project's core and kernel features.
- Constructed a foundational system to support the application.
- Implemented an audio system to enhance user experience.
- Created a central database to integrate all games into a cohesive system.
- Enabled parental oversight of children's progress and provided extensive control over game content, including questions and visuals.

PROJECTS

Quantum-Crypto

Aug 2026 – Present

Dart Package

- Engineered a post quantum cryptography suite for Flutter using Rust and providing NIST standardized ML-KEM (FIPS 203) and ML-DSA (FIPS 204) with native execution outside the Dart heap.
- Architected a hybrid key exchange layer combining X25519 with ML-KEM-512/768/1024, providing a migration path from classical public key cryptography to post quantum security.
- Implemented a complete cryptographic stack including ML-KEM, ML-DSA, AES-256-GCM, ChaCha20-Poly1305, HKDF-SHA256, streaming encryption, deterministic key derivation, and sign then encrypt secure messaging.
- Optimized cryptographic workloads through Rust execution and FFI, achieving sub-0.5ms average roundtrip latency for ML-KEM and ML-DSA operations, with up to ~19.8K HKDF operations/sec and ~165 MB/s ChaCha20-Poly1305 throughput in benchmark workloads.
- Developed security focused test coverage including cryptographic correctness, tamper detection, key serialization, hybrid exchange validation, and performance regression benchmarks across 1000-iteration workloads.

Waffle-DB

Jan 2026 – Present

Dart Package

- Engineered local vector database for Flutter, powered by Rust and HNSW graph indexing, supporting high dimensional embeddings up to 1536+ dimensions.
- Architected FFI data path between Dart and Rust, minimizing heap allocations and UI thread blocking while enabling native vector storage, retrieval, and similarity search.
- Implemented parallel vector ingestion using Rust Rayon, achieving ~24,400 vectors/sec for batch insertion and ~7,500 KNN similarity queries/sec at k=10 under benchmark workloads.
- Optimized memory and query performance through configurable HNSW parameters, quantization, cache management, and workload specific database profiles for mobile, read heavy, and write heavy applications.

Dart Package

- Engineered a medical imaging library for Flutter, utilizing Rust and FFI bridge to parse complex DICOM files without blocking the Dart UI thread, achieving an ultra fast throughput of ~296 FPS.
- Implemented custom GPU Fragment Shaders to deliver true 16bit depth precision and real time Hounsfield Unit windowing, executing over 3,400 fluid contrast adjustments per second with sub-16ms latency.
- Optimized memory allocation through FFI buffers, successfully processing, rendering, and disposing of clinical datasets containing 250+ volumetric frames in under 1 second while maintaining a highly stable 6.00ms p99 latency.
- Architected a robust and developer friendly API featuring a built in DicomViewer widget, strict edge case resilience for mathematical overflows, and seamless dependency injection to support scalable, medical applications.

Network-Reachability [↗](#)

Dec 2025 – Present

Dart Package

- Engineered a network monitoring library for Flutter, leveraging a native Rust core to execute complex multi target probing and DNS resolution without blocking the Dart UI thread.
- Developed advanced diagnostic algorithms to calculate real-time connection stability (Latency, Jitter, Packet Loss) and battery-aware adaptive polling.
- Architected resilient networking patterns including a guard () wrapper with a built in Circuit Breaker mechanism and Thundering Herd protection to optimize API requests.
- Implemented security features, enabling the detection of DNS hijacking, VPN/Proxy usage, and Captive Portals.

FP-Growth [↗](#)

Nov 2025 – Present

Dart Package

- Engineered a data mining library implementing the FP-Growth algorithm, capable of processing 1,000,000+ transactions in ~1.46s.
- Architected a memory optimized two pass system utilizing a fixed size Isolate Pool for parallel processing, enabling the analysis of massive datasets without out of memory errors.
- Developed advanced association rule generation, calculating complex statistical metrics including Support, Confidence, Lift, Leverage, and Conviction for deep data insights.
- Built a comprehensive Developer Tooling ecosystem, including a standalone CLI tool and stream based CSV adapters to parse and export large files (JSON/CSV) efficiently.

Radix-Plus [↗](#)

Oct 2025 – Present

Dart Package

- Engineered Radix Plus, a sorting library outperforming Dart's native List.sort() by up to 7.0x (~191ms for 1M+ elements using Uint32List).
- Implemented Parallel Processing utilizing Dart Isolates, unlocking massive multi-core speedups that sort 1,000,000+ elements in just ~29ms (56.5x faster).
- Optimized Memory Allocation through in-place sorting, buffer pooling, and zero-copy operations directly on TypedData to eliminate memory overhead and GC pressure.
- Developed Adaptive Algorithms providing stable sorting for int, double (with strict NaN handling), and BigInt across diverse data sizes.

OPEN SOURCE CONTRIBUTIONS

Yaru.dart [↗](#)

Mar 2026 – Present

Ubuntu Linux

- Enhanced package maintainability and developer experience by implementing strict type annotations, standardizing deprecation warnings, and upgrading core dependencies to resolve critical pub.dev [↗](#) compatibility issues.
- Streamlined the testing architecture by migrating from the base Dart test package to flutter test, successfully consolidating the testing framework and resolving fragmented Continuous Integration reports

Toastification [↗](#)

Apr 2026 – Present

Dart Package

- Optimized package efficiency and significantly reduced application bundle sizes APK/AAB by eliminating heavy, unused external dependencies

Musicpod [↗](#)

Present

Ubuntu Flutter Community

- Spearheaded the application's internationalization by implementing full Arabic localization and engineering seamless Right to Left layout support across the entire UI
- Ensured cross locale UI/UX consistency by adapting complex layouts for RTL rendering, thoroughly testing and verifying zero regressions for existing Left to Right users

EDUCATION

Benha University [↗](#)

Oct 2022 – May 2026 | Benha, Egypt

Bachelor of Computer Science

TECH SKILLS

Languages: Dart | Go | Kotlin | Python

Frameworks: Flutter | Gin

Tools & Others: Linux | Git | Docker | CI/CD

CERTIFICATES

The Complete Dart Development Guide [↗](#) | The Complete Flutter Development Guide [↗](#)

LANGUAGES

Arabic — Native

English — Fluent

Japanese — Basic